

Hotel Guest Check-in / Check-out Monitoring System

ASP.NET Core MVC Laboratory Exercise Requirements

Objective

Build a server-rendered ASP.NET Core MVC application using Model Binding, DTOs, Data Annotation Validation, Cookie Authentication, Razor Views, Bootstrap, and an in-memory Repository Pattern. No database or JavaScript is required.

Learning Outcomes

- MVC architecture
- Model binding
- DTOs and validation
- Cookie Authentication & [Authorize]
- Repository pattern
- CRUD operations
- Searching/filtering
- Bootstrap forms and tables
- Razor Tag Helpers

Business Scenario

A small hotel wants reception staff to monitor guest arrivals and departures. Staff members register their own accounts, log in, record guest check-ins, update guest information, and check guests out.

Functional Requirements

1. Register staff account with First Name, Last Name, Email, Username, Password, Confirm Password.
2. Login using Username or Email and Password.
3. Logout by clearing authentication cookie.
4. Register guest check-in.
5. View all guests in a monitoring table.
6. View guest details.
7. Edit guest information while Checked In.
8. Check out a guest by recording the current date and time.
9. Search guests by Guest Number, Name, or Room Number.

Guest Fields

- GuestNumber (auto-generated)
- FirstName

- LastName
- ContactNumber
- RoomNumber
- NumberOfGuests
- CheckInDateTime
- ExpectedCheckOutDate
- ActualCheckOutDateTime (nullable)
- Status (Checked In/Checked Out)
- Notes

Guest Monitoring Page

Display a table with Guest Number, Guest Name, Room Number, Number of Guests, Check-in Date & Time, Check-out Date & Time, Status, and Actions (Details, Edit, Check Out). Checked-out guests remain visible for historical monitoring.

Entities

User: Id, FirstName, LastName, Email, Username, Password

Guest: Id, GuestNumber, FirstName, LastName, ContactNumber, RoomNumber, NumberOfGuests, CheckInDateTime, ExpectedCheckOutDate, ActualCheckOutDateTime, Status, Notes

Repositories

Implement UserRepository and GuestRepository using static List. Controllers must not store data directly.

Controllers and Views

Controllers: HomeController, AccountController, GuestController.

Views: Login, Register, Index, Create, Edit, Details, CheckOut.

Validation

Use Data Annotations such as Required, EmailAddress, Phone, Compare, Range, Display, and DataType. Validate ModelState before processing requests.

End-to-End User Story

- Register staff account.
- Login.
- Open Guest Monitoring.
- Check in a guest.
- View guest in monitoring list.
- Edit guest if needed.
- Check out guest.
- Monitoring list now shows both check-in and check-out times.

Submission Requirements

- Complete ASP.NET Core MVC solution
- Bootstrap UI
- Cookie Authentication
- Protected guest pages
- Validation on all forms
- Model Binding
- DTOs
- Repository Pattern
- Search
- In-memory storage
- Clean code

Suggested Grading Rubric (100 pts)

Criteria	Pts
Authentication	15
CRUD	25
Validation	15
Model Binding/DTOs	10
Repository	10
Search	10
UI	10
Code Quality	5